DAY 1 · AWS LINUX LAB

Linux Users & Accounts

Admin users & service accounts

Created a dedicated sudo admin account (`adm-kiwi`) instead of using the default `ubuntu` user, then built a proper no-login service account (`svc-disk-report`) to run background scripts under least privilege.

This establishes the identity model the rest of the lab's services build on.

Local — your machine · Remote — the server · Config file · Concept · Gotcha · Best practice · Paths

What we covered

- ✓ Created `adm-kiwi` as a personal sudo admin account
- ✓ Username rules — dots (`.`) are invalid; landed on hyphenated naming
- ✓ Copied the SSH key to the new user so it could authenticate directly
- ✓ Explored `/etc/passwd` and `/etc/group` — how Linux actually stores user and group data
- ✓ Learned the "default private group" mechanism behind `adduser`
- ✓ Personal admin accounts vs. dedicated service accounts — least privilege, blast radius, auditability
- ✓ Created `svc-disk-report` as a no-login service account

Full command sequence

Remote Create the personal admin account

```
sudo adduser adm-kiwi            # create user + home directory
sudo usermod -aG sudo adm-kiwi   # -aG: append to the sudo group (keeps others)
groups adm-kiwi                  # verify group membership
```

Remote Copy the SSH key to the new user

```
sudo mkdir -p /home/adm-kiwi/.ssh                                    # -p: no error if it exists
sudo cp ~/.ssh/authorized_keys /home/adm-kiwi/.ssh/authorized_keys   # copy the allowed public key
sudo chown -R adm-kiwi:adm-kiwi /home/adm-kiwi/.ssh                  # -R: hand folder to the user
sudo chmod 700 /home/adm-kiwi/.ssh                                   # 700: only owner can enter
sudo chmod 600 /home/adm-kiwi/.ssh/authorized_keys                   # 600: owner read/write only
```

Gotcha · SSH silently refuses keys if the `.ssh` folder or `authorized_keys` is owned by root or too open — the chown and chmod steps are what make the login work.

Remote Delete a user (if starting over)

```
sudo deluser --remove-home kiwi   # --remove-home: delete user + home dir
```

Remote Inspect users & groups

```
grep adm-kiwi /etc/passwd   # user's UID / primary GID / shell
grep adm-kiwi /etc/group    # group memberships
id adm-kiwi                 # numeric UID/GID + all groups, human-readable
```

Tidier form · The original used `cat /etc/passwd | grep adm-kiwi`. `grep` can read the file itself, so the `cat` and pipe aren't needed — same result.

Remote Create a service account

```
sudo useradd -r -s /usr/sbin/nologin svc-disk-report   # -r: system acct, -s: no-login shell
```

How Linux stores accounts

Concept /etc/passwd format

```
username:x:UID:GID:comment:homedir:shell
```

The 4th field (GID) points to the user's primary group — the one auto-created by `adduser` with the same name as the user.

Concept /etc/group format

```
groupname:x:GID:member1,member2,...
```

The member list only holds additional members (added via `usermod -aG`) — not the user whose primary group this is.

Where things live

What	Path
User accounts	`/etc/passwd`
Group definitions	`/etc/group`
New user's authorized keys	`/home/<user>/.ssh/authorized_keys`
Service account shell (blocks login)	`/usr/sbin/nologin`

Best practices

Leave the default user behind
Don't work as the provider's default user (`ubuntu`) long-term — create your own admin account.

Humans and services apart
Personal sudo accounts are for you; background services get their own dedicated accounts.

Service accounts are locked down
No login shell, no sudo, no password — least privilege by design.

One account per service
Contains the blast radius and gives a clearer audit trail.

Follow the naming rule
`adm-kiwi` for personal admin, `svc-<purpose>` for services.

Build, then hand over
Build and test as your own account first, then give ownership to the service account once it works.

Part of the AWS/Linux sysadmin lab · see day1-naming-conventions for the full naming reference